

RAG COM POSTGRESQL + PGVECTOR

Objetivo

Neste capítulo, vamos entender como o projeto implementa um fluxo de RAG (Retrieval-Augmented Generation) usando documentos de diferentes clientes.

Cada cliente possui documentos como:

Contrato de seguro

Aviso de sinistro

Relatório de vistoria

documento → chunks → embeddings → PostgreSQL + pgvector → busca por similaridade → resposta

O ponto mais importante é o isolamento dos dados: uma pergunta feita pelo Cliente A não pode recuperar informações pertencentes ao Cliente B.

O que é RAG?

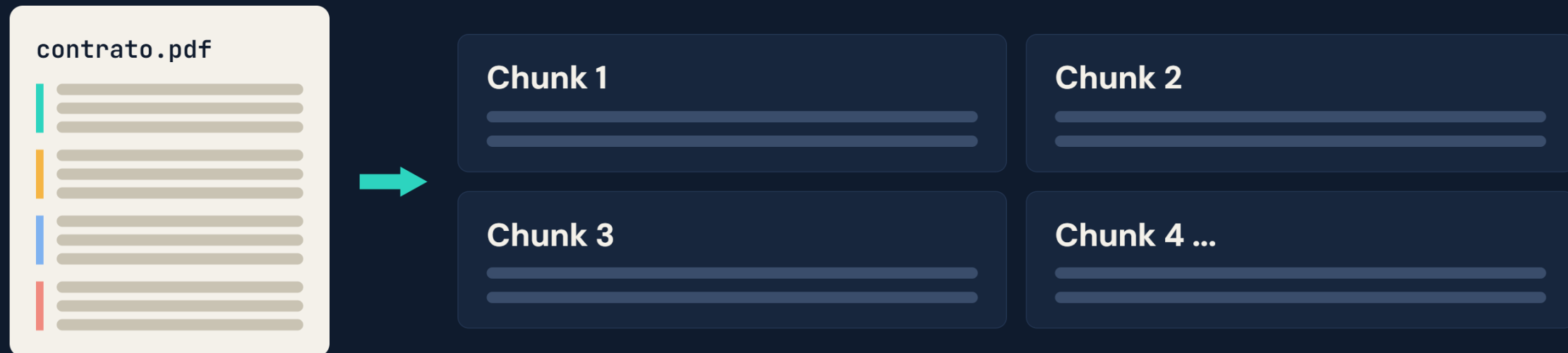
RAG significa **Retrieval-Augmented Generation**.

Em português, podemos entender como: geração aumentada por recuperação de informações. A ideia é combinar duas etapas:



No projeto, a **recuperação** acontece utilizando **embeddings + pgvector**.

O que acontece com um documento?



Imagine um PDF de contrato. O sistema **não transforma o PDF inteiro em um único embedding**: primeiro ele divide o conteúdo em partes menores, chamadas **chunks**.

No projeto, o chunk é representado pelo **TextSegment** do LangChain4j. Ele contém o texto, a ordem, o embedding e os dados de origem, como `document_id` e `client_id`.

O que é embedding?

Um embedding transforma um **texto** em uma **representação numérica**.

A ideia é que textos semanticamente relacionados produzam vetores que possam ser comparados por similaridade.

O projeto utiliza o **text-embedding-3-small** configurado com **768 dimensões**.



"O valor da franquia é R\$ 8.000"

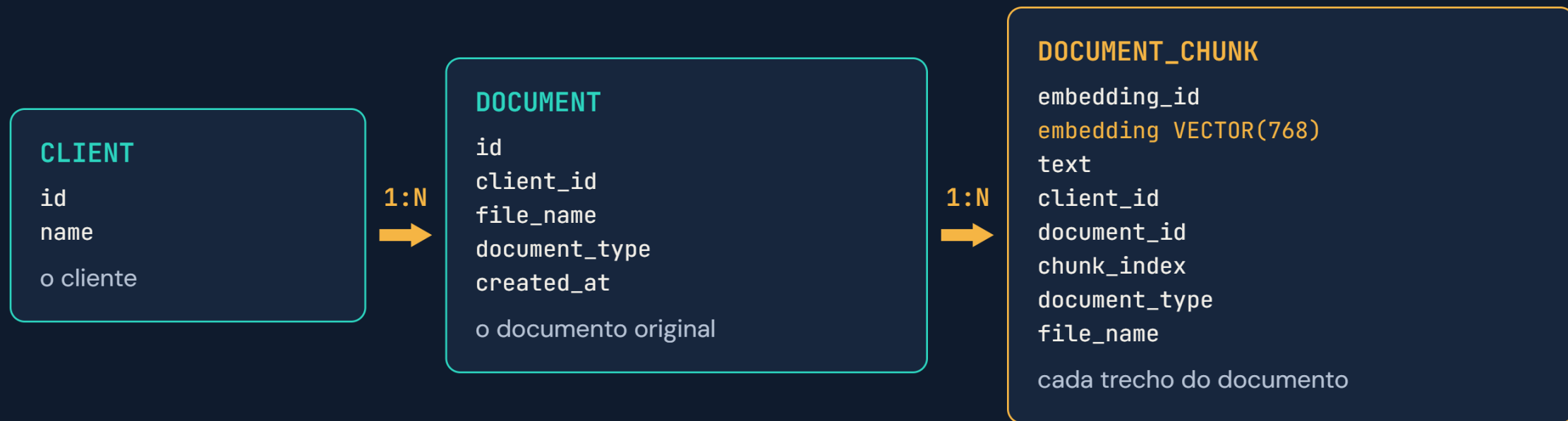
modelo de embedding

[0.12, -0.34, 0.87, ...]

Onde os embeddings ficam?

O projeto utiliza **PostgreSQL + pgvector**.

O pgvector permite armazenar e consultar vetores dentro do próprio PostgreSQL. O embedding é guardado como **VECTOR(768)**, junto do texto e do client_id.



Busca por similaridade

“Qual é o valor da franquia da minha apólice?”

A pergunta vira um embedding. O pgvector compara esse vetor com os embeddings armazenados, mas só entre os chunks do cliente autorizado.

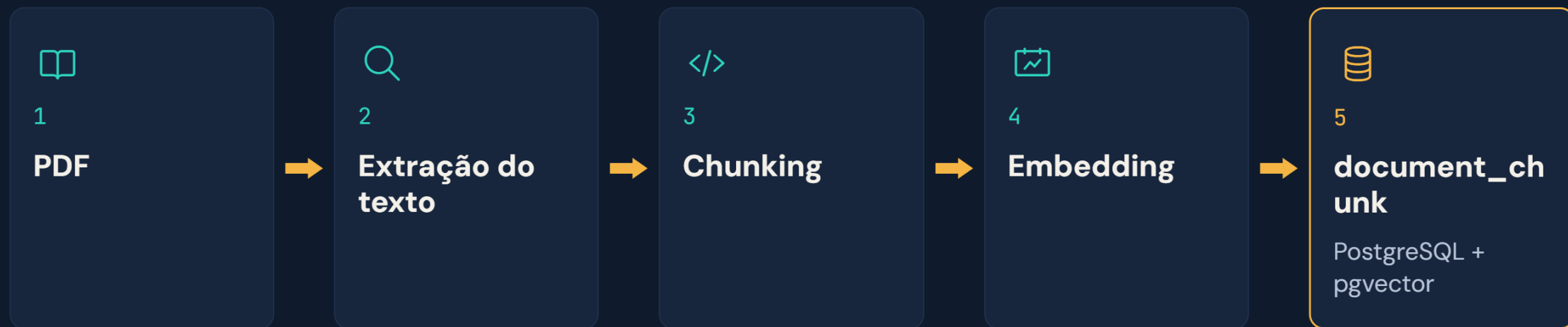
```
WHERE client_id = :clientId  
ORDER BY embedding  $\Leftrightarrow$  :queryEmbedding  
LIMIT 5;
```

Similaridade + filtro do cliente

Um índice vetorial **HNSW**, com distância de cosseno, torna essa busca mais eficiente.



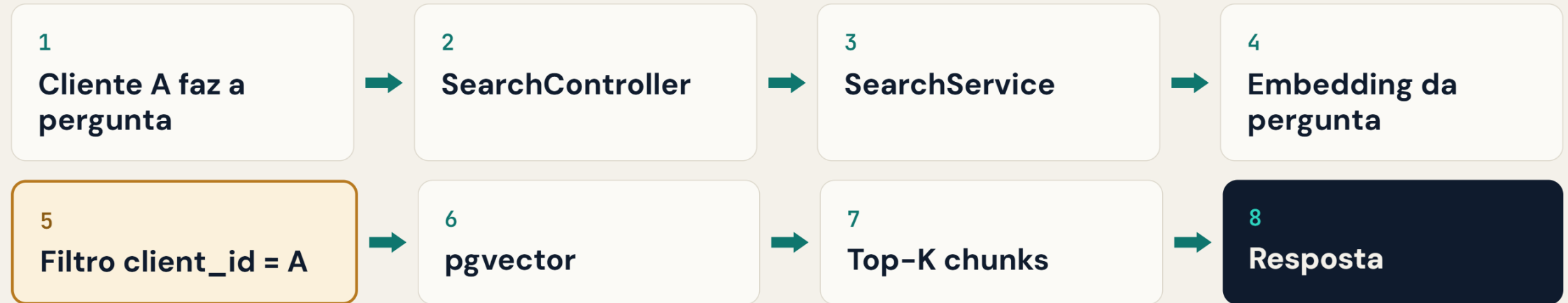
O fluxo completo de ingestão



O texto do PDF é extraído e dividido em chunks. Cada chunk recebe um embedding e é salvo na tabela **document_chunk**.

Junto com cada chunk ficam os dados de origem, como o `document_id` e o `client_id`, que permitem filtrar a busca por cliente depois.

Fluxo de consulta



O **SearchService** recebe a pergunta, gera o embedding, aplica o filtro do cliente, executa a busca vetorial e retorna os chunks mais relevantes.

O filtro do cliente entra antes do pgvector: só os chunks do cliente autorizado participam da busca.

O exemplo mais interessante do projeto

“O sinistro pode ser considerado perda total segundo as regras da apólice?”

A informação está distribuída entre documentos diferentes.

Contrato

percentual necessário para caracterizar perda total

Vistoria

custo estimado do reparo + valor do veículo



Sistema

1. recuperar as informações
2. comparar os valores
3. produzir a resposta

Tecnologias utilizadas

Backend

- Java 21
- Spring Boot
- Maven
- Spring Web
- Spring Data JPA

IA / RAG

- LangChain4j
- OpenAI
- text-embedding-3-small
- gpt-4o-mini
- PgVectorEmbeddingStore

Banco

- PostgreSQL
- pgvector

Infraestrutura

- Docker
- Docker Compose

Testes

- JUnit
- Mockito

RAG não é apenas gerar uma resposta com IA.

A qualidade e a segurança dependem de como a informação é recuperada, respeitando o cliente autorizado.